

Algoritmos y Estructuras de Datos.

- Clase 2 - Errores y Funciones -

Errores y Depuración (debugging) del código.

Cuando escribimos un programa, siempre será una tarea propensa a errores. Ya sean errores inesperados, o problemas en la estructura lógica del programa. Aquí hay algunos pasos para abordar estos problemas de manera mas sistemática:

- Comprender el problema (logico-matemáticamente), i.e. input, proceso, resultado.
- Convertir el problema abstracto en un proceso que pueda ser resuelto en un computador, i.e. escribir el algoritmo que resuelve el problema.
- Escribir el algoritmo en algun lenguaje de programación (e.g. Python).
- Comprobar y Validar que el código funcione correctamente.
- Presentación/Visualización de los resultados.

Las primeras dos etapas, obviamente tienen su complejidad y un gran potencial de introducir errores, pero por el momento no nos concentraremos en este tipo de errores. Nos enfocaremos en los errores **gramaticales** y **semánticos** generados en la 3er y 4ta etapa.

Basicamente podemos agrupar los errores en dos clases:

- **Errores Gramaticales** (tambien los llamamos **Errores Sintácticos**) no nos permiten continuar la ejecución de nuestro programa.
- **Errores Semánticos**, i.e. cuando el código continua su ejecución y terminara de manera anómala, o con resultados erróneos.

El interprete de Python detectara autamticamente los *errores sintácticos*. Los *errores semanticos* estarán en la estructura lógica del programa y será nuestro trabajo identificarlos y corregirlos.

In []:

```
# Ejemplo error de sintáxis.  
L=[ ]  
L[0] # el indice es menor a la longitud ' i > len(L)-1 '
```

In []:

```
# Ejemplo error semántico.  
L=[1,3,2,4]  
# Lsorted = L.sort()# comando incorrecto (ya que estamos cambiando la lista L)  
Lsorted = L.copy().sort() # comando correcto (ya que la lista L no ha cambiado)
```

In []:

```
# Probar ambos comandos  
print(Lsorted)
```

Errores Sintácticos

Un error sintáctico es generado cuando el interprete de Python no '*entiende*' cómo la line de código (o comando) debe ser interpretado. Cunado esto ocurre, el interprete '*lanzara*' una **excepción**. Formalmente una excepción:

- **detiene** la ejecución del cógido inmediatamente,
- demarcar qué **lineas de código** han llevado a generarse el error,
- **nombra** el error (o tipo de error),
- pasará argumentos a la excepción.

Todos los errores sintácticos son comunes, en teoría este tipo de problemas es más fácil de solucionar. Debajo podrán encontrar una breve lista de errores comunes y cómo proceder para tratarlos.

AssertionError

Esta excepción es llamada dentro del código del programa para informar un error. Veremos más en detalle, el comando `assert` , en las secciones siguientes.

In []:

```
# Ejemplo
assert not True, 'Error Aquí!!'
```

IndentationError

Esta excepción aparece cuando nuestro código no respeta la indentación de Python ("*unexpected indent*"). También aparece, cuando no hay un bloque de código luego de un ':' ("*expected an indented block*").

In []:

```
# Ejemplo
3+2 # bien!
  3+2 # mal indentado.
```

In []:

```
# Ejemplo

for x in range(0,9): # No hay ningún
                    # bloque de código
print(x) # la función print no respeta la indentación adecuada para estar dentro
del for
```

Qué podemos hacer?

- Chequear que todas las líneas de código estén alineadas correctamente. Si utilizamos algún tipo de IDE, o editor de texto avanzado, podremos ver claramente los niveles de indentación.
- En el caso de que la indentación de nuestro código sea correcta, e igualmente persiste el error **IndentationError** puede ser debido a 'mezclar' distintos tipos de *espacios y tabulaciones*. Debemos unificar estos *caracteres invisibles* de acuerdo con la indentación predefinida de Python.
- Hay un bloque de código (apropiadamente indentado) luego de los dos puntos? En el caso que querramos forzar un bloque de código 'vacío', deberemos escribir **pass** luego de los dos puntos.
- Ser cuadadoso cuando copiamos/pegamos líneas que tienen una indentación distinta.

In []:

```
# Ejemplo
for x in range(0,9): pass
print(3)
```

IndexError

Esta excepción aparece cuando el índice de un *string* o *lista* está fuera del rango.

In []:

```
# Ejemplo
L=[0,1,2]
L[3]
```

Qué podemos hacer?

- Recordar que los índices de una lista L (string s) permitidos, pertenecen al rango $[0, \dots, \text{len}(L) - 1]$ ($[0, \dots, \text{len}(s) - 1]$).
- Las listas son objetos mutables, debemos ser concientes que algunos comandos pueden extender o acortar la lista.
- Si utilizamos distintas variables para indexar, debemos comprobar que no hayan sido cambiadas previamente en nuestro código, o en alguna función externa.

NameError

Esta excepción aparece cuando una variable no es definida.

In []:

```
# Ejemplo
# notassigned =1
notassigned
```

Qué podemos hacer?

- Localizar la variable utilizando el número de línea provisto por el error/excepción.
- Localizar la línea de código (si es que existe) en donde dicha variable fue definida, es esta línea antes o despues del error? Es esta línea bien indentada?
- Revisar si hay algún error ortográfico o sintáctico al llamar la variable.
- Si el error esta dentro de una función, es posible que dicha variable es *local* y su uso es *global* (or reversamente).
- Si el nombre pertenece a una función, primero asegurarse que estemos importando el módulo que contiene la función. Lo mismo para constantes globales, por ejemplo, π or γ .

OverflowError

Esta excepción aparece cuando sobrepasamos el límite computacional de los números de punto flotante (*pueden usar solo una cantidad de memoria limitada*). En contraste los números enteros pueden ser de cualquier extensión, más precisamente hasta que nos quedamos sin memoria reservada y se lanza una excepción **MemoryError**.

In []:

```
# Ejemplo
(1.0*10**1000)/10**1000 # OverflowError!
```

Qué podemos hacer?

- Convertir números de punto flotante a enteros cuando sea posible.
- Evitar el uso de números de punto flotante muy grandes, se deben usarse distintas técnicas para obtener dicha precisión.

RuntimeError

Esta excepción aparece cuando hay un error que no tenga un tipo de error predefinidos. Es un error en ejecución '*generico*'

In []:

```
# Ejemplo
raise
```

SyntaxError

Este error aparece cuando hay un problema sintáctico en el código. Puede suceder cuando usamos caracteres especiales, intentamos asignar un valor a un nombre reservado o protegido, o a un número. Además de aparecer cuando nos olvidamos los dos puntos luego de un **for**, **if**, **while**, **def**, etc. En la documentación oficial de Python se hace referencia a estos como **Errores**, cualquier otro tipo de error una **Excepción**.

In []:

```
# Ejemplos
print(x)

for x in range(0,9)
    print(x)
```

```
File "<ipython-input-19-73923ac40357>", line 4
    for x in range(0,9)
    ^
SyntaxError: invalid syntax
```

Qué podemos hacer?

- Utilizar letras sin acento u otros caracteres especiales.
- Cuando escribimos `1 = ...`, siempre, lo que queríamos escribir es: `1 == ....` Chequear que no hayamos confundido estos símbolos `=` y `==` en ninguna de nuestras expresiones.
- Recordar escribir los dos puntos al terminar la línea de un `for` , `if` , `while` , `def` , etc.

TypeError

Esta excepción aparece cuando utilizamos un tipo erróneo, es decir distinto al esperado. Ocurre usualmente cuando llamamos a una función la cual requiere de tipos de parámetros específicos. Un número erróneo de argumentos. Y por otras diversas razones relacionadas a conflictos de tipos.

In []:

```
# Ejemplos
L["1"]
#len(3)
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-22-b77a5574dc57> in <module>
      1 # Ejemplos
      2 #L["1"]
----> 3 len(3)

TypeError: object of type 'int' has no len()
```

Que podemos hacer?

- Buscar nombre de variables duplicados. Intentar escribir nombres mas largos en caso de ser necesario.
- Siempre podemos utilizar la función `help` para ver información sobre la cantidad y tipos de argumento que una función requiere.
- Para el caso de los métodos, podemos obtener esta información llamando a `help` sobre el tipo del objeto.

ValueError

Esta excepción aparece cuando llamamos a una función (o método) con argumentos del tipo correcto, pero con valores que no son validos para dicha función (método).

In []:

```
# Ejemplo
int('value')
#1/0
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
<ipython-input-26-9f9fe7cc4db0> in <module>
      1 # Ejemplo
      2 #int('value')
----> 3 1/0

ZeroDivisionError: division by zero
```

Qué podemos hacer?

- Llamar a la función `help` y verificar que los valores son validos.
- Cuando intentamos dividir por cero, obtenemos directamente una excepción **ZeroDivisionError**. Estar atentos a aquellos números de punto flotante que se aproximen a cero.
- En general, estos problemas son específicos caso por caso, en caso de querer circunventarlos podemos utilizar comandos `if` , o bloques de código `try` . Por ejemplo, estas dos formas nosotros mismos realizamos el manejo del error a priori, o a posteriori respectivamente.

UnboundLocalError

Esta excepción aparece cuando el interprete reconoce el uso de una variable local con alcance global. En la siguiente Lección definiremos en detalle el *alcance de una variable*.

Errores Semánticos

Los errores semánticos son mucho mas frecuentes que los sintácticos. Por definición, son errores del tipo **el programa hace algo, pero no lo que quiero**. En este caso, el programa esta libre de problemas sintacticos graves, o es posible ejecutarlo "sin errores".

Una solución general para detectar problemas semánticos es monitorear los valores de las variables durante la ejecución del programa, ya sea utilizando numerosos lineas `print` , o utilizando alguna herramienta de depuración (debugger) que nos facilite esta tarea.

Sintoma : A veces obtengo un error sintático.

Primero debemos revisar el error acorde a lo visto para cada caso de error sintático. Sin embargo, algunos errores sintáticos aparecen porque el programador ha realizado un error semántico. Aquí tenemos una breve lista de posibles problemas a chequear:

- Existen asignaciones a variables de objetos mutables (usando =)? Entonces, deberíamos reemplazarlas usando el método `copy()`. Recordar que los objetos mutables, son objetos dinámicos.
- Asegurarse que cuando definimos `a = función(...)`, la función `función` retorne un valor.
- Entender el comportamiento de las funciones, en especial para evitar parametros con valores invalidos (e.g. listas vacias, tipos incompatibles, etc).

In []:

```
# Ejemplo
L=[1,32,3]
M = L # M y L son/apuntan a la misma lista.
print("L=",L,"M=",M)
L.pop(1)
print("L=",L,"M=",M)
M[2]
# Probar reemplazando por M = L.copy(). Funcionan de la misma forma?
```

In []:

```
# Ejemplo
L=[1,3,2,4]
Lsorted=L.sort() # Además del efecto secundario que el metodo sort tiene, es ord
enar la lista L 'sobre si misma'.
```

In []:

```
# Explicar por que obtenemos un error. Corregir el código.
L=[i for i in range(0,10)] # Queremos obtener una lista de enteros impares.
for x in range(0,5):
    print(L)
    L.pop(2*x) # '2*x' hace referencia a los indices pares en la lista original
               (los números impares)
               # pero estamos cambiando la lista en cada paso del ciclo.
```

Sintoma : Mi programa no termina su ejecución.

Una vez comenzada la ejecución de nuestro programa, a veces, parece que no fuera a terminar de ejecutarse nunca. Debemos tratar de asegurarnos que podemos identificar cuales con las líneas de código reponsables. Existen dos casos posibles, el código no terminara nunca de ejecutarse, o eventualmnte terminara su ejecución, pero no sabremos cuando.

Cómo difereciamos ambos casos?

Ejecutar el programa con la menor cantidad de datos posibles y comprobar que el código funciona correctamente.

el programa no termina

- Corroborar que la condiciones de los ciclos **while** terminen eventualmente.
- Si estamos utilizando una llamada recursiva en una función, corroborar que exista un caso base el cual asegure la terminación de las llamadas recursivas.
- Estamos modificando los indices en un ciclo meintras que lo recorremos (en su interior)?

el programa es "muy pesado"

- Estamos realizando grandes calculos? Podemos evitarlos? Existen módulos especiales optimizados para este tipo de calculos?
- Estamos usando las estructuras de datos correctas?
- Utilizamos funciones propias para realizar el trabajo de funciones ya definidas?
- Estimar la complejidad del problema y como su complejidad crece de acuerdo al tamaño de los datos.
- No hacer abuso de funciones recursivas (si no están extremadamente optimizadas).

In []:

```
# Código que nunca terminara.  
L=[0]  
for x in L:  
    print(x)  
    L.append(x)  
  
# Otro ejemplo.  
#while True:  
#    print(1)
```

Sintoma : Mi programa no retorna valores correctos.

En este momento nuestro programa se ejecuta sin errores, pero retorna resultados erroneos. La pregunta es: **Cuán mal estan los resultados?** Si el resultado es de un tipo erroneo, podemos suponer algun error de casting o *TypeError* dentro de nuestro código. Aquí tenemos una breve lista de posibles problemas a chequear:

- Separar el codigo en funciones o módulos, para tratar de encapsular el error, y poder testear distintas partes del código de manera individual.
- Chequear los objetos que sean mutables.

In []:

```
# Ejemplo
mess=list(set(mess)) # ordenara eficientemente la lista 'mess', pero multiple va
lores pueden perderse.
```

Manejo de Excepciones

Los errores en nuestro código terminan la ejecución del programa. En algunos casos, nos gustaría ignorar algunos errores no críticos, y poder continuar ejecutando el programa. Utilizando el comando **try** podemos capturar excepciones (errores) y decidir como proseguir. Para 'lanzar' una excepción, es decir, informar o propagar un error, utilizaremos los comandos **raise** y **assert**

El comando Try

Está es la forma de utilizar el comando **try** :

try :

```
{bloque A de código con potenciales errores}
{bloque A de código con potenciales errores}
```

except *Error1* **as** *string1* :

```
{bloque B1 de código para el manejo del *Error1*}
{bloque B1 de código para el manejo del *Error1*}
```

except *Error2* **as** *string2* :

```
{bloque B2 de código para el manejo del *Error2*}
{bloque B2 de código para el manejo del *Error2*}
```

else :

```
{bloque C de código para ejecutar finalmente, si no hay errores}
{bloque C de código para ejecutar finalmente, si no hay errores}
```

Funciona de la siguiente forma, primero se ejecutara el bloque de código A, el cual contendrá comandos que podrían generar errores del tipo *Error1* o *Error2*. Si hay ningún error, a continuación se ejecutará el bloque de código C luego del **else** . Si una de las excepciones/errores (contenidos en la lista de errores), el bloque de código B1 o B2, respectivamente al error. Cabe destacar que todo el código del bloque A debajo del error, así como el bloque de código C no serán ejecutados. El parámetro *string1* contine los argumentos o informe del error.

In []:

```
L=[1,2]
L[2]=6
```

In []:

```
# Ejemplo
try:
    L=[1,2]
    L[2]=6
except IndexError:
    print(' Hay un problema!')
else: pass
```

También es posible una tupla de errores al comando **except**, en vez de escribirlos separados **except Error1 ...; except Error2 ...**, etc. Una 'tupla de errores', solo debe ser usada para agrupar aquellas excepciones/errores que manejaremos (o ignoremos) de la misma forma. En caso de que **except** no tenga un *Error* como parámetro, este será activado por '*cualquier error*' (todos los tipos de error).

El comando Assert

El comando **assert** es un comando simple, muy útil para depurar nuestro código. Lo usamos de la siguiente forma:

assert *una condición, un string*

Si la condición es verdadera (**True**), el código continuara su ejecución normalmente. En caso de que la condición sea falsa (**False**), se generara un error/excepcion del tipo **AssertionError** y se imprimira el argumento *string* en la consola. Si la *expresión* de la condición no es del tipo booleano, igualmente se evaluara y luego se intentara convertir el resultado a tipo booleano (de ser posible).

In []:

```
# Ejercicio: Que hace este código?
import random
mess=[]
if random.randrange(0,2)==1:
    N=20
    for p in range(0,N):
        mess.append(random.randrange(0,100000))

# Creara una lista de 20 números aleatorios, o una lista vacia, con probabilidad 1/2.
```

In []:

```
# Ejercicio: Escribir un programa que imprima
# el maximo de la lista 'mess', llamada max(mess), y el indice correspondiente:
# "max(mess)=mess[i]"
# si la lista esta vacía, informar un 'assert error'.
mess=[1,2,3,4]
assert len(mess)>0,'la lista está vacía' # El mensaje de error del comando assert
assert mess[2] != 3,'indice fuera de rango'

maxelement=0
for x in mess:
    if x>maxelement:
        maxelement=x
Lindices=[]
for i in range(0,len(mess)):
    if maxelement==mess[i]:
        Lindices.append(i)
```

Funciones

Para definir una función usamos la palabra reservada `def`, luego escribimos la '*signatura*' de la función, esto es, el *nombre* de dicha función y los *parámetros* que recibe. Como siempre, debemos termina la línea con `:` para indicar que a continuación habrá un bloque de código. Obviamente, luego respetar la indentación.

Definition

Este es un prototipo típico de la signatura de una función:

```
assert nombreFunción(arg_1,...,arg_r ):

    {código de la función}
    ...
    {más código de la función}
```

Al igual que cualquier bloque de código indentado, debe contener al menos una línea, en otro caso deberemos escribir `pass` luego de los `:'`.

Algunas reglas básicas para definir funciones :

- El *Nombre* de la función debe ser representativo (al igual que con los nombres de variable).
- Puede aceptar tantos argumentos como sea necesario, incluso ninguno.
- Siempre que llamamos una función debemos añadir los *parentesis* `()`, aunque la función no reciba parámetros.
- Si no proveemos la cantidad correcta de argumentos, la función retornará un error del tipo **TypeError**.
- Los argumentos no tienen un tipo definido. Para controlar que los argumentos sean del tipo esperado deberemos, tendremos que validar su tipo y en caso de error, utilizar `assert` e informar un *nuevo* tipo de error **TypeError** propio.
- Para que la función retorne un valor `v` usaremos, usaremos el comando `return (v)`.
- Luego del comando `return` es ejecutado, la función termina.
- Si no usamos el comando `return` en el cuerpo de la función, entonces el valor de la función del tipo **NoneType**.

In []:

```
# Cuál es la diferencia entre estas dos funciones?
def Helloworld1():
    print("Hello world")
def Helloworld2():
    return(12,"Hello world")
    print(3)
```